Scoping Application-Permission Mailbox Access — Graph RBAC & Application Access Policies for Exchange Online

Implementation Solution Guide

Version 1.0.0 · 2026-07-17

Contents

Executive Summary — Scoping Application-Permission Mailbox Access (Graph RBAC & Application Access Policies) 4

What you get . . . 4

What this delivers . . . 4

The standard deployment model . . . 4

Outcome . . . 4

1. Solution Overview — Scoping Application-Permission Mailbox Access (Graph RBAC & Application Access Policies) 5

The exposure: an application permission is tenant-wide by default . . . 5

The two scoping models . . . 5

The decision path (which model fits) . . . 6

How the boundary is proven . . . 6

Benefits . . . 6

Limitations (be honest with buyers) . . . 7

The standard deployment model . . . 7

2. Requirements Guide — Scoping Application-Permission Mailbox Access 7

2.1 An Exchange Online organization . . . 8

2.2 An admin role that can create access policies / assign Exchange RBAC . . . 8

2.3 An app registration or managed identity holding the application permission . . . 8

2.4 A mail-enabled security group (for the Application Access Policy model) . . . 8

2.5 A scope object (for the Graph RBAC for Exchange Online model) . . . 9

2.6 Exchange Online PowerShell + Microsoft Graph PowerShell . . . 9

2.7 Change control, a pilot app, and rollback . . . 9

Checklist . . . 9

Global Requirements — Azure Solutions 10

G1. A Microsoft 365 / Entra tenant . . . 10

G2. Administrative roles (least privilege) . . . 10

G3. Licensing baseline . . . 11

G4. Network connectivity to Microsoft 365 / Azure endpoints . . . 11

G5. Change control & maintenance window . . . 11

G6. Backup / rollback posture . . . 11

3. Implementation Guide — Scoping Application-Permission Mailbox Access 12

Naming (set once, reuse verbatim) . . . 12

Decision path — choose the model FIRST . . . 13

Step 1 — Discover and confirm the starting state . . . 13

Model A — Application Access Policy . . . 13
Model B — Graph RBAC for Exchange Online (resource-scoped) . . . 14
Step 7 — Validate the boundary (both models) . . . 15
Step 8 — Expand to the next app . . . 15
Rollback (for every change) . . . 15
5. Testing Guide — Scoping Application-Permission Mailbox Access 15
Test 1 — The app is scoped, not tenant-wide . . . 16
Test 2 — Microsoft's built-in policy test (Model A) . . . 16
Test 3 — In-scope mailbox read SUCCEEDS . . . 16
Test 4 — Out-of-scope mailbox read is DENIED . . . 16
Test 5 — Evidence captured for auditors . . . 16
Test 6 — Rollback restores prior state (change-safety check) . . . 17
Pass/fail gate . . . 17
6. Troubleshooting Guide — Scoping Application-Permission Mailbox Access 17
A. The policy seems to have no effect / propagation delay . . . 17
B. The app still reads ALL mailboxes (out-of-scope NOT denied) . . . 18
C. The IN-scope read is DENIED (scope too tight) . . . 18
D. Group membership not honored / wrong group type . . . 19
E. Graph RBAC for Exchange Online — assignment errors . . . 19
F. Scripts — connection, scopes, exact-name matching, -WhatIf . . . 19
G. When to escalate . . . 20
7. Best Practices — Scoping Application-Permission Mailbox Access 20
8. Deliverables Summary — Scoping Application-Permission Mailbox Access 21
Documentation . . . 21
PowerShell scripts . . . 22
What the buyer receives . . . 22
Outcome . . . 22
Appendix A — PowerShell Scripts 23

Executive Summary — Scoping Application-Permission Mailbox Access (Graph RBAC & Application Access Policies)

White-label · Azure Solutions

Granting an application — an app registration or the managed identity of a Logic App or Function — an **application** Graph permission over mail (`Mail.Read`, `Mail.Send`, `Mail.ReadWrite`, `MailboxSettings.*`) is **tenant-wide by default**: it can read, write, or send as **every mailbox in the organization**, not just the one it was built to service. That is almost never intended, and it is a classic security-review finding — the blast radius of a leaked secret or a compromised managed identity is the entire organization's mail.

This solution implements **least-privilege mailbox access** for apps and managed identities: it scopes the identity to a specific mailbox or a mail-enabled security group of allowed mailboxes, and then **proves** the boundary. It is written for an administrator with no prior experience of Exchange application access policies or Exchange RBAC, and it is the standalone reference that the sibling automation solutions (Logic Apps mailbox automation, managed-identity authentication to Microsoft 365) point to for scoping.

What you get

- A plain-language **Solution Overview** of the exposure and the **two scoping models** — Application Access Policy and Graph RBAC for Exchange Online — with a clear decision path for which fits.
- A **Requirements Guide** with verify / implement-if-missing for every prerequisite.
- An **Implementation Guide** covering **both** models end to end — create the mail-enabled security group and bind the app (Model A), or create the resource scope and assign a scoped Exchange role (Model B) — every object named and reused, every cmdlet flagged to verify against current Microsoft documentation.
- Five **PowerShell scripts** for discovery, scoping validation, a tenant-wide audit report, the in-scope/out-of-scope boundary proof, and a `-WhatIf`-gated policy setter.
- **Testing** and **Troubleshooting** guides with explicit PASS/FAIL gates.

What this delivers

An application's mailbox access **scoped** to exactly the mailboxes it is allowed to touch, with a **tested, auditable boundary**: the in-scope read succeeds, the out-of-scope read is **denied**, and the evidence is captured for auditors — using whichever of the two Exchange scoping models fits your estate.

The standard deployment model

Assess → **Configure** → **Pilot** → **Validate** → **Expand** — with the boundary proof (in-scope PASS, out-of-scope DENIED) as the gate and a documented rollback.

Outcome

Least-privilege mailbox access for every app and managed identity — a scope proven to hold, both models explained, and a tenant-wide report driving the count of unscoped mail permissions to zero — instead of a

tenant-wide grant that can reach every mailbox in the organization.

1. Solution Overview — Scoping Application-Permission Mailbox Access (Graph RBAC & Application Access Policies)

The exposure: an application permission is tenant-wide by default

When you grant an **application** (an app registration, or the managed identity of a Logic App / Function / VM) an **application-type** Microsoft Graph permission over mail — for example `Mail.Read`, `Mail.ReadWrite`, `Mail.Send`, `MailboxSettings.Read`, or their Exchange-Web-Services equivalents — that grant is **tenant-wide**. The app can read, write, or send as **every mailbox in the organization**, not just the one mailbox it was built to service. This is different from *delegated* permissions (which act only as the signed-in user); an **application permission runs with no user and is bounded by nothing** unless you add a boundary yourself.

That is almost never what the app actually needs, and it is a classic security-review finding: an invoicing automation that should read one shared mailbox can, with the same credential, read the CEO's inbox. The blast radius of a leaked client secret or a compromised managed identity is *the entire organization's mail*.

> ***This solution's whole premise is least privilege.*** *A tenant-wide application mail permission with no scope is **not** an acceptable end state — it is the finding you are here to close. Every path below narrows the app to exactly the mailboxes it is allowed to touch, and then **proves** the boundary.*

The two scoping models

Exchange Online provides **two** ways to constrain an application's mailbox access. This product owns the deep material for **both**, because they solve the same problem differently and the right choice depends on your situation.

- **Application Access Policy** — the established model. You create a **mail-enabled security group** containing the mailboxes the app is allowed to touch, then bind the app to that group with an access policy (using `New-ApplicationAccessPolicy` in Exchange Online PowerShell — *verify the exact cmdlet name and parameters against current Microsoft documentation*). The policy restricts (or denies) the app to the group's members. It layers a **scope on top of** the broad application permission the app already holds.

- **Graph RBAC for Exchange Online** — the newer, resource-scoped model (sometimes described as **RBAC for Applications in Exchange Online**; *confirm the current product name against Microsoft documentation*). Instead of granting a broad Graph application permission and then restricting it, you assign the app's **service principal** a specific Exchange **application role** (for example an "Application Mail.Read"-style role — *verify exact role names against current Microsoft documentation*) **scoped** to a management scope, administrative unit, or group of mailboxes. Access is granted **narrowly from the start** rather than granted broadly and then fenced in.

> ***Which model is current / recommended is version- and roadmap-dependent — do NOT assume.*** *Microsoft has been evolving the RBAC-for-applications model. **Verify against current Microsoft documentation** which model Microsoft recommends for a new deployment, whether both are supported in your cloud (commercial vs GCC / GCC High / DoD), and the exact cmdlets, role names,*

*and scope objects before you implement. This guide gives you the workflow for each and a decision path; it deliberately does **not** assert which one Microsoft currently prefers.*

The decision path (which model fits)

Use this as a starting decision, then confirm against current Microsoft guidance:

1. **Do you already operate Application Access Policies, and is your app holding a broad Graph mail permission you simply need to fence to a set of mailboxes?** → **Application Access Policy** is the low-friction fit: create the mail-enabled security group, bind the app, done.

2. **Do you want the app to hold only a narrow, specific Exchange right scoped to specific mailboxes — never a tenant-wide grant at all — for the tightest possible least privilege?** → lean toward **Graph RBAC for Exchange Online** (resource-scoped assignment), if it is available and supported for your scenario and cloud.

3. **Is the app a managed identity (Logic App / Function)?** Both models can scope a managed identity's service principal; the mechanics are the same as for an app registration. *Verify managed-identity support for the specific model and cloud.*

4. **Are you in a government cloud (GCC / GCC High / DoD)?** Feature parity and rollout timing differ from commercial — **verify** that the chosen model and its cmdlets are available in your cloud before committing.

When unsure, pilot the **Application Access Policy** model first (it is the more broadly available and the one most audit reviewers recognize), and evaluate Graph RBAC as the target-state model once you have verified its support.

How the boundary is proven

Scoping is only credible if you can **demonstrate** it. This solution proves the boundary two ways:

• **In-scope read succeeds** — the app, using its own application credential, reads the mailbox it is allowed to touch (`reports@contoso.com`). This must **PASS**.

• **Out-of-scope read is denied** — the same app, same credential, attempts to read a mailbox it is *not* scoped to (`ceo@contoso.com`). This must be **DENIED** (a `403`/`ErrorAccessDenied`-class failure). If an out-of-scope read succeeds, the scope is not in effect — the product's troubleshooting guide covers exactly this.

The evidence (who the app is, which mailboxes it can and cannot reach, and the deny result) is captured to a report so an auditor can see the boundary without re-running anything.

Benefits

• **Least privilege, provable.** The app touches only its intended mailboxes, and you have the test evidence to show it.

• **Smaller blast radius.** A leaked secret or compromised managed identity can reach one mailbox, not the whole tenant.

• **Two models, one reference.** Whether your estate uses Application Access Policies or is moving to Graph RBAC for Exchange Online, the decision path and the boundary proof are the same.

• **Audit-ready.** A tenant-wide report of every mail-capable application permission and whether it is scoped is exactly what a security review asks for.

• **Reusable across every app/identity.** This scoping applies to any application or managed identity that reads or sends mail — including the automation patterns covered by the sibling solutions (Logic Apps mailbox automation, managed-identity authentication to Microsoft 365).

Limitations (be honest with buyers)

• **A scope constrains the app, not the permission grant.** With Application Access Policy, the app still *holds* the broad Graph permission; the policy fences it. If the policy is removed or misconfigured, the broad grant is back in force — the report and validation script exist to catch that.

• **Propagation is not instant.** New Application Access Policies and RBAC assignments take time to take effect across Exchange Online (commonly cited as up to ~30 minutes or longer — *verify current propagation guidance with Microsoft*). A boundary test run too soon can give a false result.

• **Model availability varies.** Graph RBAC for Exchange Online is newer; its exact cmdlets, role names, supported resources, and availability in government clouds are **version-dependent — verify before relying on them.**

• **Groups must be the right type.** Application Access Policy requires a **mail-enabled security group** — a plain security group or a Microsoft 365 group will not behave the same way; membership must be maintained as mailboxes come and go.

• **This is access scoping, not DLP.** Scoping limits *which mailboxes* an app can reach; it does not inspect or restrict *what content* leaves. Pair it with data-loss-prevention controls where the risk warrants.

The standard deployment model

> ***Discover mail-capable application permissions and whether each is scoped → choose the model (Application Access Policy or Graph RBAC for Exchange Online) → create the scope (mail-enabled security group / management scope) and bind the app → validate the app is scoped (not tenant-wide) → prove the boundary (in-scope PASS, out-of-scope DENIED) → expand to the next app.*** *The repeatable pattern:* ***Assess → Configure → Pilot → Validate → Expand****, with the* ***boundary proof as the gate*** *and a documented rollback (remove the policy / assignment) for every change.*

2. Requirements Guide — Scoping Application-Permission Mailbox Access

Verify **every** requirement before configuring. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites are in `global-requirements-azure-solutions.md`. Objects are named `<Purpose> – <Scope> – <Stage>` and reused verbatim.

> ***Verify-with-Microsoft note:*** *exact* ***role names****,* ***cmdlet names/parameters****,* ***Graph permission names****, and* ***which scoping model is current/recommended*** *are version- and cloud-dependent. Where a requirement names one, treat it as a starting point and* ***confirm against current Microsoft***

documentation *— do not assume.*

2.1 An Exchange Online organization

Description: A Microsoft 365 tenant with **Exchange Online** and the mailboxes the app will (and will not) be allowed to access. **Why:** Both scoping models — Application Access Policy and Graph RBAC for Exchange Online — are Exchange Online features; there is nothing to scope without Exchange Online mailboxes. **Verify:** `https://admin.exchange.microsoft.com` resolves and lists mailboxes; run `MBXRBAC-Discovery.ps1`. **Implement-if-missing:** License and provision Exchange Online mailboxes for the in-scope and out-of-scope test identities (`reports@contoso.com`, `ceo@contoso.com`).

2.2 An admin role that can create access policies / assign Exchange RBAC

Description: An operating account holding an Exchange admin role sufficient to run `New-ApplicationAccessPolicy` **and/or** to create Exchange RBAC scopes and role assignments — commonly **Global Administrator** or an Exchange role such as one carrying the **"Role Management"** / **"Organization Configuration"** rights (*verify the exact minimum role names against current Microsoft documentation*). **Why:** Creating an Application Access Policy, and creating a management scope / role assignment for Graph RBAC, are privileged Exchange operations. **Verify:** In Exchange admin center → **Roles** → **Admin roles**, confirm the account's role; `MBXRBAC-Discovery.ps1` reports the connected identity. **Implement-if-missing:** Assign the minimum role that grants the needed rights at the narrowest scope; prefer PIM eligible activation over standing access.

2.3 An app registration or managed identity holding the application permission

Description: The **application identity** to be scoped — an Entra **app registration** (with a client ID / `AppId`) or the **managed identity** of a Logic App / Function — that holds a mail-capable **application** Graph permission (e.g. `Mail.Read`). In this guide the example identity is `Mailbox Reporting App` (`AppId` `11111111-1111-1111-1111-111111111111`, an example value — substitute your own). **Why:** The scope binds to a specific application by its `AppId` / service-principal object ID; without an existing application identity there is nothing to scope. **Why it is the exposure:** the mail-capable **application** permission it holds is tenant-wide until scoped — this is exactly what you are fencing. **Verify:** Entra → **App registrations** (or **Enterprise applications** for the service principal / managed identity) → **API permissions** shows the granted application permission with admin consent; `MBXRBAC-Discovery.ps1` lists apps/identities holding mail-capable application permissions and whether each is scoped or tenant-wide. **Implement-if-missing:** Register the app (or enable the managed identity), grant **only** the specific mail application permission the workload needs (least privilege at the permission level too), and grant admin consent. *Verify the exact Graph permission name required for the operation against current Microsoft documentation.*

2.4 A mail-enabled security group (for the Application Access Policy model)

Description: A **mail-enabled security group** whose members are exactly the mailboxes the app is allowed to touch. Named `App Mailbox Scope – Reporting – Prod` (primary SMTP `app-mailbox-scope-reporting@contoso.com`). **Why:** `New-ApplicationAccessPolicy` scopes an app to a **mail-enabled security group** — not a plain security group and not a Microsoft 365 group. The

group's membership *is* the allow-list. **Verify:** Exchange admin center → **Recipients** → **Groups** shows `App Mailbox Scope – Reporting – Prod` as a **mail-enabled security group**; `MBXRBAC-Validate.ps1` confirms the group type and membership. **Implement-if-missing:** Create the mail-enabled security group and add only the allowed mailbox(es) (`reports@contoso.com`). *Verify the exact group type requirement against current Microsoft documentation — only mail-enabled security groups are supported by Application Access Policy.* (Not required if you use the Graph RBAC model exclusively.)

2.5 A scope object (for the Graph RBAC for Exchange Online model)

Description: The **resource scope** the RBAC assignment will be limited to — a management scope, administrative unit, or group of mailboxes that defines which mailboxes the assigned Exchange application role may reach. Named `App Mailbox Scope – Reporting – RBAC Scope`. **Why:** Graph RBAC for Exchange Online grants a specific Exchange application role **scoped** to a resource; the scope object is what makes the grant narrow instead of tenant-wide. **Verify:** Confirm the scope object exists and resolves to the intended mailboxes (*verify the exact scope object type and cmdlet — e.g. management scope vs administrative unit — against current Microsoft documentation*). **Implement-if-missing:** Create the scope object bounding it to the allowed mailbox(es); use `MBXRBAC-Set-AccessPolicy.ps1` (which defaults to `-WhatIf`) to preview the assignment. (Not required if you use the Application Access Policy model exclusively.)

2.6 Exchange Online PowerShell + Microsoft Graph PowerShell

Description: The **ExchangeOnlineManagement** module (for `Connect-ExchangeOnline`, access policies, and Exchange RBAC) and the **Microsoft.Graph** module (for discovering and reporting application permissions and running the boundary read). *Verify current module names and minimum versions against current Microsoft documentation.* **Why:** Every step and script in this solution runs in one or both modules; the access-policy and RBAC cmdlets live in ExchangeOnlineManagement, the app-permission discovery and Graph mail read live in Microsoft.Graph. **Verify:** `Get-Module -ListAvailable ExchangeOnlineManagement, Microsoft.Graph`; `MBXRBAC-Discovery.ps1` checks and imports both. **Implement-if-missing:** `Install-Module ExchangeOnlineManagement -Scope CurrentUser` and `Install-Module Microsoft.Graph -Scope CurrentUser`.

2.7 Change control, a pilot app, and rollback

Description: An approved change window, a named **pilot application** (`Mailbox Reporting App`) and pilot mailboxes, and a documented rollback before you scope any production app. **Why:** A mis-scoped policy can break a working automation (in-scope reads start failing) or leave the app tenant-wide; the pilot and rollback are the safety net. **Verify:** An approved change record, the pilot app/mailboxes, and a written back-out exist (per `global-requirements-azure-solutions.md` G5/G6). **Implement-if-missing:** Capture current state with `MBXRBAC-Report.ps1`, define the pilot app + in-scope/out-of-scope mailboxes, and write the rollback (remove the Application Access Policy / RBAC assignment).

Checklist

- Exchange Online organization + in-scope/out-of-scope mailboxes — `MBXRBAC-Discovery.ps1`

• Admin role that can create access policies / assign Exchange RBAC (verify exact role)

• App registration or managed identity holding the mail application permission (`Mailbox Reporting App`)

• Mail-enabled security group `App Mailbox Scope – Reporting – Prod` (Application Access Policy model)

• Scope object `App Mailbox Scope – Reporting – RBAC Scope` (Graph RBAC model)

• ExchangeOnlineManagement + Microsoft.Graph modules installed

• Change window, pilot app/mailboxes, and rollback documented

Global Requirements — Azure Solutions

These requirements are common to **every** Azure Solutions solution in the standard program model. They are the baseline platform prerequisites that any Azure/Microsoft 365 engagement assumes. Each is structured **Description / Why / Verify / Implement-if-missing**. Solution-specific requirements (see `02-requirements.md`) are layered on top of these.

> *Naming convention: every created object is named `<Purpose> – <Scope> – <Stage>` and reused verbatim throughout the engagement.*

G1. A Microsoft 365 / Entra tenant

• **Description:** An active Microsoft 365 / Microsoft Entra tenant for the cloud(s) in scope (commercial and/or government).

• **Why:** All identity, licensing, and service configuration is anchored to a tenant; nothing can be configured without one.

• **Verify:** Sign in to the relevant admin center (`https://admin.microsoft.com`) and confirm the tenant resolves and is active under **Settings** → **Org settings**.

• **Implement-if-missing:** Provision the tenant through the appropriate channel (commercial: standard; government: via an authorized AOS-G reseller).

G2. Administrative roles (least privilege)

• **Description:** Appropriately scoped admin roles for the work — Global Administrator for tenant-wide changes, and service-specific roles (Exchange, SharePoint, Teams, Azure Owner/Contributor) for day-to-day.

• **Why:** Every configuration and migration action requires a role with sufficient privilege; least-privilege scoping limits blast radius and satisfies audit requirements.

• **Verify:** In Entra → **Roles & admins**, confirm the operating accounts hold the required roles. In Azure → **Subscriptions** → **Access control (IAM)**, confirm RBAC assignments.

• **Implement-if-missing:** Assign the minimum roles needed; prefer eligible (PIM) activation over standing access where available.

G3. Licensing baseline

• **Description:** Sufficient Microsoft 365 and/or Azure licensing/credit for all in-scope users and workloads.

• **Why:** Features and capacity (mailboxes, SharePoint storage, AVD, VMs) are gated by license SKUs and subscription quota.

• **Verify: Billing** → **Licenses** for M365; **Subscriptions** → **Usage + quotas** for Azure. Compare assigned/available against the in-scope object count.

• **Implement-if-missing:** Procure the correct SKUs and request quota increases sized to the workload (add headroom for shared/resource objects).

G4. Network connectivity to Microsoft 365 / Azure endpoints

• **Description:** Reliable network egress to the required Microsoft 365 and Azure service endpoints (and the sovereign `.us` endpoints for government clouds), with sufficient bandwidth.

• **Why:** All cloud administration and any data movement depends on connectivity to the correct endpoint set; government clouds use distinct endpoints from commercial.

• **Verify:** Confirm reachability to the relevant endpoints (commercial `*.microsoft.com` / government `*.microsoft.us`, `*.azure.us`, `*.office365.us`). Measure sustained bandwidth against the data volume.

• **Implement-if-missing:** Open firewall/proxy allowances for the correct endpoint set; scale bandwidth or add ExpressRoute (matching the cloud — Gov ExpressRoute for Gov regions).

G5. Change control & maintenance window

• **Description:** An approved change-management process with scheduled maintenance windows and a stakeholder communications plan.

• **Why:** Cloud changes can be outward-facing and disruptive; controlled windows and comms reduce business impact and satisfy governance/compliance.

• **Verify:** Confirm an approved change record and a scheduled window for each disruptive action.

• **Implement-if-missing:** Stand up a lightweight change/approval process; schedule windows; prepare user comms templates.

G6. Backup / rollback posture

• **Description:** A defined backup and rollback position for any change — config export, data backup, and a documented back-out procedure before any disruptive action.

• **Why:** Cloud changes are not always reversible by default; a tested rollback is the safety net for cutovers and bulk operations.

• **Verify:** Confirm pre-change config/state is captured and a written rollback exists for each disruptive change.

• **Implement-if-missing:** Export current configuration/state; document and (where feasible) rehearse the rollback before executing the change.

3. Implementation Guide — Scoping Application-Permission Mailbox Access

Follow these steps **in order**. Steps run in **Exchange Online PowerShell** (`Connect-ExchangeOnline`) and **Microsoft Graph PowerShell** (`Connect-MgGraph`), with portal confirmations in the **Exchange admin center** (`https://admin.exchange.microsoft.com`) and **Microsoft Entra admin center** (`https://entra.microsoft.com`). Objects are named `<Purpose> – <Scope> – <Stage>` and reused verbatim in every later step, script, and test.

> ***Verify-with-Microsoft note (applies to every cmdlet, role name, permission name, and model name below):*** *exact cmdlet names, parameters, Exchange role names, Graph permission names, scope-object types, and* ***which of the two models Microsoft currently recommends*** *are version- and cloud-dependent.* ***Confirm each against current Microsoft documentation before running it.*** *The names below are a faithful starting point, not an assertion of current fact.*

Naming (set once, reuse verbatim)

Object	Name	Notes
Pilot application (the identity being scoped)	`Mailbox Reporting App`	`AppId` `11111111-1111-1111-1111-111111111111` (example — substitute yours)
In-scope mailbox (allowed)	`reports@contoso.com`	the app **may** read this
Second in-scope mailbox (allowed)	`shared-invoices@contoso.com`	optional; also allowed
Out-of-scope mailbox (must be denied)	`ceo@contoso.com`	the app **must not** read this
Mail-enabled security group (Model A)	`App Mailbox Scope – Reporting – Prod`	SMTP `app-mailbox-scope-reporting@contoso.com`
Application Access Policy (Model A)	`App Mailbox Scope – Reporting – Prod – Policy`	the policy description
Scope object (Model B)	`App Mailbox Scope – Reporting – RBAC Scope`	management scope / admin unit
RBAC role assignment (Model B)	`App Mailbox Scope – Reporting – RBAC Assignment`	the scoped role assignment

Decision path — choose the model FIRST

Decide before you configure (then confirm against current Microsoft guidance — see the note above):

1. **App already holds a broad Graph mail permission and you just need to fence it to a set of mailboxes, and/or your estate already uses Application Access Policies** → do **Model A (Application Access Policy)** — Steps 2–4.

2. **You want the app to hold only a narrow, specific Exchange right scoped to specific mailboxes (no tenant-wide grant at all) — the tightest least privilege** → do **Model B (Graph RBAC for Exchange Online)** — Steps 5–6, *if it is available and supported for your scenario and cloud*.

3. **Unsure, or in a government cloud** → pilot **Model A** first (more broadly available and audit-recognized); verify **Model B** availability before adopting it as target state.

4. **Either way**, always finish with **Step 7 (validate)** and the **Testing Guide** (prove the boundary).

Step 1 — Discover and confirm the starting state

1. Open PowerShell and connect: `Connect-ExchangeOnline` and `Connect-MgGraph -Scopes "Application.Read.All","Directory.Read.All","Mail.Read"` (*verify the exact scopes required against current Microsoft documentation*).

2. Run `MBXRBAC-Discovery.ps1` — it lists every app / managed identity holding a mail-capable **application** permission (`Mail.Read`, `Mail.ReadWrite`, `Mail.Send`, `MailboxSettings.*`, etc.) and whether each is **scoped** (has an access policy / RBAC assignment) or **tenant-wide**.

3. Confirm `Mailbox Reporting App` (`AppId` `11111111-1111-1111-1111-111111111111`) appears as **tenant-wide** — this is the exposure you are closing.

4. Run `MBXRBAC-Report.ps1` to capture the pre-change state as your rollback baseline.

Model A — Application Access Policy

Step 2 — Create the mail-enabled security group

1. In **Exchange Online PowerShell**, create the mail-enabled security group and name it exactly `App Mailbox Scope – Reporting – Prod`:

```
New-DistributionGroup -Name "App Mailbox Scope – Reporting – Prod" `
  -Type Security `
  -PrimarySmtpAddress "app-mailbox-scope-reporting@contoso.com"
```

(Verify `New-DistributionGroup -Type Security` is the current way to create a mail-enabled security group against current Microsoft documentation.)

1. Add **only** the allowed mailbox(es) as members:

```
Add-DistributionGroupMember -Identity "App Mailbox Scope – Reporting – Prod" -Member "reports@contoso.c
```

Add `shared-invoices@contoso.com` too if it is in scope. **Do not** add `ceo@contoso.com`.

1. Confirm membership in Exchange admin center → **Recipients** → **Groups** → `App Mailbox Scope – Reporting – Prod`.

Step 3 — Create the Application Access Policy binding the app to the group

1. Create the policy that restricts `Mailbox Reporting App` to the group:

```
New-ApplicationAccessPolicy `
  -AppId "11111111-1111-1111-1111-111111111111" `
  -PolicyScopeGroupId "app-mailbox-scope-reporting@contoso.com" `
  -AccessRight RestrictAccess `
  -Description "App Mailbox Scope – Reporting – Prod – Policy"
```

(Verify the exact cmdlet name `New-ApplicationAccessPolicy`, the `-AccessRight` values — e.g. `RestrictAccess` vs `DenyAccess` — and all parameters against current Microsoft documentation. `RestrictAccess` limits the app to the group; `DenyAccess` blocks the app from the group.)

1. Note the `-Description` value **is** the policy's name for later reference: `App Mailbox Scope – Reporting – Prod – Policy`.

Step 4 — Verify the policy took effect (allow propagation)

1. **Wait for propagation** before testing — new access policies are not instant (commonly cited up to ~30 minutes; *verify current guidance with Microsoft*).

2. Test the boundary with Microsoft's built-in test (*verify the exact cmdlet name against current Microsoft documentation*):

```
Test-ApplicationAccessPolicy -Identity "reports@contoso.com" -AppId "11111111-1111-1111-1111-1111111111
Test-ApplicationAccessPolicy -Identity "ceo@contoso.com"     -AppId "11111111-1111-1111-1111-1111111111
```

1. Run `MBXRBAC-Validate.ps1` (PASS/FAIL: the app is scoped, not tenant-wide) and proceed to **Step 7** then the **Testing Guide**.

Model B — Graph RBAC for Exchange Online (resource-scoped)

> *Do this model **instead of** Model A when you want the app to hold only a narrow, specific Exchange right scoped to specific mailboxes. **Verify every cmdlet, role name, and scope-object type below against current Microsoft documentation — this model is newer and its exact syntax and availability (especially in government clouds) are version-dependent.***

Step 5 — Create the resource scope

1. Register the application's **service principal** in Exchange (if the RBAC model requires an Exchange-side service principal object) — *verify the exact cmdlet (e.g. `New-ServicePrincipal`) and whether it is required against current Microsoft documentation*:

```
New-ServicePrincipal -AppId "11111111-1111-1111-1111-111111111111" -ObjectId "<SP object id>" -DisplayN
```

1. Create the **resource scope** that bounds access to the allowed mailbox(es), and name it `App Mailbox Scope – Reporting – RBAC Scope` — *verify the exact scope-object type and cmdlet (management scope vs administrative unit vs group filter) against current Microsoft documentation*:

```
New-ManagementScope -Name "App Mailbox Scope – Reporting – RBAC Scope" `
  -RecipientRestrictionFilter "MemberOfGroup -eq '<DN of App Mailbox Scope – Reporting – Prod>'"
```

Step 6 — Assign the scoped Exchange application role

1. Assign the specific Exchange **application role** (e.g. an "Application Mail.Read"-style role — *verify the exact role name against current Microsoft documentation*) to the app's service principal, scoped to the resource scope, and name the assignment `App Mailbox Scope – Reporting – RBAC Assignment`:

```
New-ManagementRoleAssignment -Name "App Mailbox Scope – Reporting – RBAC Assignment" `
  -App "11111111-1111-1111-1111-111111111111" `
```

```
    -Role "Application Mail.Read" `
    -CustomResourceScope "App Mailbox Scope – Reporting – RBAC Scope"
```

(Verify the exact cmdlet `New-ManagementRoleAssignment`, the `-App` / `-Role` / scope parameters, and the exact role name against current Microsoft documentation.)

1. **Wait for propagation**, then run `MBXRBAC-Validate.ps1` and proceed to **Step 7** and the **Testing Guide**.

Step 7 — Validate the boundary (both models)

1. Run `MBXRBAC-Validate.ps1` → confirms `Mailbox Reporting App` is **scoped** (an Application Access Policy or an RBAC assignment covers it) and **not** tenant-wide.

2. Run `MBXRBAC-Test-Boundary.ps1` → proves the app can read the **in-scope** mailbox (`reports@contoso.com` — **PASS**) and is **denied** the **out-of-scope** mailbox (`ceo@contoso.com` — **DENIED**).

3. Run `MBXRBAC-Report.ps1` → captures the post-change scoping evidence for auditors. Complete the **Testing Guide** before declaring the app scoped.

Step 8 — Expand to the next app

1. Repeat the chosen model for the next application / managed identity, giving each its **own** mail-enabled security group / scope object and its own policy / assignment — never reuse one app's scope for another.

2. Re-run `MBXRBAC-Report.ps1` across the tenant each cycle; the goal is **zero** mail-capable application permissions left tenant-wide.

Rollback (for every change)

• **Model A:** `Remove-ApplicationAccessPolicy` for `App Mailbox Scope – Reporting – Prod – Policy` (*verify cmdlet*) — the app returns to its prior (tenant-wide) state; existing mail is unaffected.

• **Model B:** `Remove-ManagementRoleAssignment "App Mailbox Scope – Reporting – RBAC Assignment"` (*verify cmdlet*).

• If in-scope reads start failing after scoping, check group membership / scope filter and propagation before removing the policy (see the Troubleshooting Guide).

Order of operations

`Discover (is the app tenant-wide?) → choose model → [Model A: group → policy → verify] OR [Model B: scope → role assignment → verify] → validate scoped → prove boundary (in PASS / out DENIED) → expand`

5. Testing Guide — Scoping Application-Permission Mailbox Access

Run after the scope is configured and **after propagation has completed** (Implementation Step 4 for Model A / Step 6 for Model B). Both scoping models are validated the same way — the boundary behaviour is identical, only the mechanism differs. Objects are referenced by their `<Purpose> – <Scope> – <Stage>` names.

> ***Propagation:*** *a new Application Access Policy or RBAC assignment is **not** instant. Testing too early gives a false result (an out-of-scope read may still succeed for a short window). Allow the propagation time (commonly cited up to ~30 minutes; *verify with Microsoft*) before trusting a DENIED/allowed result.*

Test 1 — The app is scoped, not tenant-wide

Do: Run `MBXRBAC-Validate.ps1`; or confirm the policy/assignment exists (Model A: `Get-ApplicationAccessPolicy`; Model B: the RBAC role assignment — *verify cmdlets*). **Expected:** `Mailbox Reporting App` (`AppId` `11111111-1111-1111-1111-111111111111`) is covered by `App Mailbox Scope – Reporting – Prod – Policy` **or** `App Mailbox Scope – Reporting – RBAC Assignment`. The app is reported **scoped**, not tenant-wide. **PASS/FAIL:** PASS = a scope covers the app. FAIL = the app is still tenant-wide.

Test 2 — Microsoft's built-in policy test (Model A)

Do: `Test-ApplicationAccessPolicy -Identity "reports@contoso.com" -AppId "11111111-1111-1111-1111-111111111111"` and again with `-Identity "ceo@contoso.com"` (*verify cmdlet name*). **Expected:** `reports@contoso.com` → **Granted/AccessCheckResult = Granted**; `ceo@contoso.com` → **Denied**. **PASS/FAIL:** PASS = in-scope Granted **and** out-of-scope Denied. FAIL = either result is wrong.

Test 3 — In-scope mailbox read SUCCEEDS

Do: Run `MBXRBAC-Test-Boundary.ps1` (it authenticates **as the application** and reads `reports@contoso.com`); or manually call Graph `GET /users/reports@contoso.com/messages?$top=1` with an **application** token for the app. **Expected:** The read **succeeds** (HTTP 200, at least the message metadata is returned). **PASS/FAIL:** PASS = in-scope read returns 200. FAIL = in-scope read is denied (scope too tight / wrong group membership — see Troubleshooting).

Test 4 — Out-of-scope mailbox read is DENIED

Do: With the **same** application token, read `ceo@contoso.com` (Graph `GET /users/ceo@contoso.com/messages?$top=1`); `MBXRBAC-Test-Boundary.ps1` does this automatically. **Expected:** The read is **denied** — HTTP **403** with an `ErrorAccessDenied`-class error (the app is not scoped to this mailbox). **PASS/FAIL:** PASS = out-of-scope read returns 403/denied. **FAIL = out-of-scope read succeeds** — the scope is not in effect (wrong model applied, mis-scope, or propagation not complete). This is the critical failure the whole solution exists to prevent.

Test 5 — Evidence captured for auditors

Do: Run `MBXRBAC-Report.ps1` and save the CSV. **Expected:** A tenant report showing every mail-capable application permission and, for each, whether it is scoped and to which mailboxes — with `Mailbox Reporting App` shown scoped to `App Mailbox Scope – Reporting – Prod` / the RBAC scope. **PASS/FAIL:** PASS = the report shows the app scoped and names the allowed mailboxes; the boundary is auditable without re-running a live read.

Test 6 — Rollback restores prior state (change-safety check)

Do: In a non-production rehearsal, remove the policy/assignment (Model A: `Remove-ApplicationAccessPolicy`; Model B: `Remove-ManagementRoleAssignment` — *verify cmdlets*), re-run `MBXRBAC-Validate.ps1`, then re-apply. **Expected:** After removal the app reports **tenant-wide** again (proving the scope was what constrained it); after re-apply it reports **scoped**. **PASS/FAIL:** PASS = removal and re-apply both behave as expected, confirming a working rollback.

Pass/fail gate

Gate	Requirement
Scoped	Test 1 passes — a policy/assignment covers `Mailbox Reporting App`; it is not tenant-wide
Boundary proven	Tests 3 **and** 4 pass — in-scope read **succeeds**, out-of-scope read is **DENIED (403)**. Both are required
Auditable	Test 5 passes — the scoping report names the app and its allowed mailboxes
Reversible	Test 6 passes — rollback restores prior state cleanly

Do not declare an app scoped unless Test 4 (out-of-scope DENIED) passes. A passing in-scope read alone does not prove least privilege.

6. Troubleshooting Guide — Scoping Application-Permission Mailbox Access

Use `MBXRBAC-Discovery.ps1` / `MBXRBAC-Validate.ps1` for scoping state, `MBXRBAC-Report.ps1` for the tenant-wide picture, `MBXRBAC-Test-Boundary.ps1` for the live boundary proof, and `MBXRBAC-Set-AccessPolicy.ps1` (defaults to `-WhatIf`) to (re)create the scope. Objects are referenced by their `<Purpose> – <Scope> – <Stage>` names.

> ***Verify-with-Microsoft note:*** *every cmdlet, role name, permission name, propagation window, and the choice of scoping model named below is version- and cloud-dependent —* ***confirm against current Microsoft documentation*** *before acting.*

A. The policy seems to have no effect / propagation delay

Symptom	Cause	Remediation
Boundary test still lets the app read every mailbox right after you create the policy	**Propagation not complete** — new Application Access Policies / RBAC assignments are not instant (commonly cited up to ~30 minutes or longer; *verify with Microsoft*)	Wait the propagation window, then re-run `MBXRBAC-Test-Boundary.ps1`. Don't record a result until propagation completes
`Test-ApplicationAccessPolicy` shows the right answer but a live read does not	Cached tokens for the app still carry the old (unscoped) grant	Wait for the app's token to expire / re-acquire, then re-test

B. The app still reads ALL mailboxes (out-of-scope NOT denied)

This is the critical failure — the boundary is not in effect. Work through, in order:

1. **Wrong model applied for the situation.** Confirm which model you intended (Application Access Policy vs Graph RBAC for Exchange Online) and that it is the one actually configured. An RBAC assignment does not show up as an Application Access Policy and vice-versa — `MBXRBAC-Discovery.ps1` checks the access-policy path; **verify the RBAC path separately against current Microsoft documentation**.

2. **Policy not bound to this AppId.** Run `MBXRBAC-Validate.ps1`; if it reports "no policy covers the AppId", the policy references a different app. Re-create with the correct `-AppId` (`MBXRBAC-Set-AccessPolicy.ps1`).

3. `-AccessRight` **is the wrong sense.** `RestrictAccess` limits the app **to** the group; `DenyAccess` blocks the app **from** the group. Using `DenyAccess` with an allow-list group produces the opposite of what you want. *Verify the `-AccessRight` values against current Microsoft documentation.*

4. **Propagation** — see section A.

5. **The out-of-scope mailbox is a member of the scope group** — see section D.

C. The IN-scope read is DENIED (scope too tight)

Symptom	Cause	Remediation
`MBXRBAC-Test-Boundary.ps1` fails the in-scope read (403)	The allowed mailbox is **not** a member of `App Mailbox Scope – Reporting – Prod`	Add it: `Add-DistributionGroupMember -Identity "App Mailbox Scope – Reporting – Prod" -Member "reports@contoso.com"`, allow propagation, re-test
In-scope read denied even though membership is correct	The app is missing the **permission** itself (e.g. `Mail.Read` not granted/consented)	Grant + admin-consent the specific mail application permission in Entra → the app → API permissions; *verify the exact permission name*

Symptom	Cause	Remediation
RBAC model: in-scope denied	The scoped **role** does not include the needed right, or the scope excludes the mailbox	Re-check the Exchange role and the resource scope filter; *verify role/scope cmdlets with Microsoft*

D. Group membership not honored / wrong group type

- **Application Access Policy requires a mail-enabled security group.** A plain security group or a Microsoft 365 group will not scope correctly. `MBXRBAC-Validate.ps1` flags the type; recreate as `New-DistributionGroup -Type Security` if wrong (*verify current cmdlet*).
- **Membership drift** — a mailbox added after the policy was created is only in scope once it is a **member** of `App Mailbox Scope – Reporting – Prod`. Keep membership current; re-run `MBXRBAC-Report.ps1` to see the allowed set.
- **Nested groups** may not be evaluated the way you expect — prefer direct membership; *verify nesting behaviour with Microsoft*.

E. Graph RBAC for Exchange Online — assignment errors

Symptom	Cause	Remediation
`New-ManagementRoleAssignment` / `New-ManagementScope` not recognized or errors	Cmdlet name/parameters differ by version, or the model is not available in this cloud	**Verify the exact cmdlets, role names, and scope-object types against current Microsoft documentation**; confirm availability for your cloud (GCC / GCC High / DoD) before relying on it
Assignment created but app still tenant-wide	The service principal object, the role, or the scope is mismatched	Confirm the app's Exchange service principal (`New-ServicePrincipal` if required — *verify*), the role name, and that the scope resolves to the intended mailboxes
Not sure which model to use	Availability/recommendation is roadmap-dependent	Pilot the Application Access Policy model first (broadly available, audit-recognized); adopt Graph RBAC as target state only after verifying support

F. Scripts — connection, scopes, exact-name matching, -WhatIf

```
# Connect as needed:
Connect-ExchangeOnline -ShowBanner:$false
Connect-MgGraph -Scopes 'Application.Read.All','Directory.Read.All'   # add 'Mail.Read' where a delegated

# Exact-name matching: object names contain an en dash (–), not a hyphen (-). Quote them verbatim:
Get-DistributionGroup -Identity "App Mailbox Scope – Reporting – Prod"
```

```
# The state-changer defaults to preview. It changes nothing unless you pass -Apply:
.\MBXRBAC-Set-AccessPolicy.ps1 -AppId "11111111-1111-1111-1111-111111111111"          # WhatIf preview
.\MBXRBAC-Set-AccessPolicy.ps1 -AppId "11111111-1111-1111-1111-111111111111" -Apply    # executes

# Boundary proof authenticates AS THE APP (client credentials or managed identity), not as you.
.\MBXRBAC-Test-Boundary.ps1 -TenantId "<tenant>" -AppId "<appid>" -UseManagedIdentity
```

- **Exact-name matching:** the mail-enabled security group, policy, scope, and assignment names use an **en dash (–)**. Copy names verbatim from the Implementation Guide; a hyphen (`-`) will not match.
- **National clouds:** `MBXRBAC-Test-Boundary.ps1` targets `graph.microsoft.com`. For GCC High / DoD use the correct sovereign Graph endpoint — *verify the endpoint against current Microsoft documentation*.

G. When to escalate

If the correct model is configured against the correct `AppId`, the scope group is a mail-enabled security group with the right membership (or the RBAC scope resolves correctly), the propagation window has elapsed, and the out-of-scope read **still** succeeds, capture `MBXRBAC-Report.ps1` output and the policy/assignment details and open a Microsoft support case — a boundary that will not enforce after correct configuration and propagation is a service-side issue.

7. Best Practices — Scoping Application-Permission Mailbox Access

- **Treat a tenant-wide app mail permission as a finding, not a default.** An application `Mail.Read`/`Mail.Send`/etc. grant reaches **every** mailbox until you scope it. Never ship an app or managed identity with an unscoped mail application permission — scope it, then prove the boundary.
- **Least privilege at the permission level too.** Before you even scope *which mailboxes*, grant only the specific mail permission the workload needs (read-only `Mail.Read`, not `Mail.ReadWrite`, unless it truly writes). Scoping the mailbox set and minimizing the permission compound. *Verify exact permission names against current Microsoft documentation.*
- **Choose the model deliberately, then verify it.** Application Access Policy (fence a broad grant to a mail-enabled security group) and Graph RBAC for Exchange Online (grant a narrow right scoped to a resource) solve the same problem differently. Pick per the decision path — and **confirm which model Microsoft currently recommends, and that it is available in your cloud, against current documentation** before committing.
- **Name everything and reuse the name.** Use `<Purpose> – <Scope> – <Stage>` for the group, policy, scope object, and role assignment, and reuse it verbatim across scripts, tests, and rollback so the boundary stays auditable and cleanable.
- **One app, one scope.** Give each application/identity its **own** mail-enabled security group or resource scope. Never share one scope across apps — it blurs the boundary and makes least privilege un-provable.
- **Prove the boundary both ways, every time.** In-scope read must **succeed** *and* out-of-scope read must be **DENIED (403)**. A passing in-scope read alone is not evidence of least privilege — the deny is the proof.

• **Wait for propagation before you trust a test.** New policies/assignments take time to take effect; a test run too soon can show an out-of-scope read still succeeding. Allow the propagation window (*verify current guidance with Microsoft*) before recording a result.

• **Use a mail-enabled security group — and maintain it.** Application Access Policy requires a **mail-enabled security group**; plain security groups and Microsoft 365 groups do not behave the same. Keep membership current as mailboxes are added or retired.

• **Report tenant-wide, remediate to zero.** Run the scoping report across the whole tenant each cycle; the target is **no** mail-capable application permission left unscoped. The CSV is the audit artifact.

• **Keep a rollback for every change.** Removing the Application Access Policy or the RBAC assignment returns the app to its prior state; write the rollback down before you apply the scope, and rehearse it once.

• **This scopes access, not content.** Mailbox scoping limits *which mailboxes* an app can reach; pair it with DLP where you also need to control *what* content can leave.

8. Deliverables Summary — Scoping Application-Permission Mailbox Access

Documentation

Document	Purpose
Executive Summary	The exposure (tenant-wide app mail permission), what's in the box, and the outcome
1. Solution Overview	The exposure, the **two scoping models** (Application Access Policy & Graph RBAC for Exchange Online), when each applies, the decision path, how the boundary is proven, benefits, limitations
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing (Exchange Online, admin role, app/managed identity, mail-enabled security group, scope object, PowerShell modules)
Global Requirements (Azure Solutions)	Category-wide prerequisites, published once
3. Implementation Guide	Decision path up front, then exact ordered steps for **both** models — Model A (group → access policy) and Model B (scope → scoped role assignment) — every object named, every cmdlet flagged verify-with-Microsoft
5. Testing Guide	Boundary proof with PASS/FAIL gates: in-scope read succeeds, out-of-scope read DENIED, evidence captured
6. Troubleshooting Guide	Field issues — no effect / propagation, still reads all mailboxes, out-of-scope not denied, group membership, RBAC errors

Document	Purpose
7. Best Practices	Field-proven recommendations

PowerShell scripts

Script	What it does
`MBXRBAC-Discovery.ps1`	Read-only — lists apps/managed identities holding mail-capable application permissions (Mail.Read/Mail.Send/etc.) and whether each is **scoped** or **tenant-wide**
`MBXRBAC-Validate.ps1`	PASS/FAIL — confirms the target app/identity is **scoped** (not tenant-wide) to the intended mailbox(es)
`MBXRBAC-Report.ps1`	Reports every app-permission-to-mailbox scoping (or lack of it) across the tenant + CSV — the audit artifact
`MBXRBAC-Test-Boundary.ps1`	Read-only — proves the in-scope read works and the out-of-scope read is denied (the boundary proof)
`MBXRBAC-Set-AccessPolicy.ps1`	The only state-changer — creates/verifies the Application Access Policy (or RBAC assignment); **defaults to** `-WhatIf`

What the buyer receives

- This implementation guide as a **Word** document and **PDF**.
- All five **PowerShell scripts** in a single ZIP.
- A repeatable process following the standard model: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** — with the **boundary proof as the gate** and a documented rollback.

Outcome

Every application and managed identity that reads or sends mail is scoped to exactly the mailboxes it is allowed to touch — least privilege, with both scoping models explained and a tested, auditable boundary — instead of a tenant-wide permission that can reach every mailbox in the organization.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

MBXRBAC-Discovery.ps1

Purpose: Read-only discovery for the "Scoping Application-Permission Mailbox Access" solution. Enumerates every service principal (app registration / managed identity) granted an application-type mail permission — Mail.Read, Mail.ReadWrite, Mail.Send, MailboxSettings.*, and their EWS/other equivalents — and reports, for each, whether an Exchange Application Access Policy (or an Exchange RBAC assignment) scopes it, or whether it is still tenant-wide (able to reach EVERY mailbox). Makes no changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\MBXRBAC-Discovery.ps1 powershell -ExecutionPolicy Bypass -File .\MBXRBAC-Discovery.ps1 -AppId "11111111-1111-1111-1111-111111111111" Requires the Microsoft.Graph and ExchangeOnlineManagement modules. Run Connect-MgGraph and Connect-ExchangeOnline first, or the script will prompt. REQUIRED SCOPES / ROLES Microsoft Graph (delegated): Application.Read.All, Directory.Read.All (read app permissions). Exchange Online: a role that can run Get-ApplicationAccessPolicy (e.g. View-Only roles). VERIFY the exact Graph scopes, Exchange role names, and the app-role/permission IDs against current Microsoft documentation — they are version- and cloud-dependent. EXPECTED RESULTS A per-app snapshot: each app/identity holding a mail-capable application permission, the permission(s) it holds, and a SCOPED / TENANT-WIDE verdict. Tenant-wide apps are the exposure to close. Read-only; no policy, group, or RBAC object is created or changed. .NOTES Azure Solutions — read-only. Reports on configuration/identifiers only; makes no changes. White-label sample values use contoso.com. All cmdlet/permission names: verify against current Microsoft documentation. > [CmdletBinding()] param([string]$AppId, [string]$CsvPath)

How to run: powershell -ExecutionPolicy Bypass -File .\MBXRBAC-Discovery.ps1 powershell -ExecutionPolicy Bypass -File .\MBXRBAC-Discovery.ps1 -AppId "11111111-1111-1111-1111-111111111111" Requires the Microsoft.Graph and ExchangeOnlineManagement modules. Run Connect-MgGraph and Connect-ExchangeOnline first, or the script will prompt. REQUIRED SCOPES / ROLES Microsoft Graph (delegated): Application.Read.All, Directory.Read.All (read app permissions). Exchange Online: a role that can run Get-ApplicationAccessPolicy (e.g. View-Only roles). VERIFY the exact Graph scopes, Exchange role names, and the app-role/permission IDs against current Microsoft documentation — they are version- and cloud-dependent. EXPECTED RESULTS A per-app snapshot: each app/identity holding a mail-capable application permission, the permission(s) it holds, and a SCOPED / TENANT-WIDE verdict. Tenant-wide apps are the exposure to close. Read-only; no policy, group, or RBAC object is created or changed. .NOTES Azure Solutions — read-only. Reports on configuration/identifiers only; makes no changes. White-label sample values use contoso.com. All cmdlet/permission names: verify against current Microsoft documentation. > [CmdletBinding()] param([string]$AppId, [string]$CsvPath)

MBXRBAC-Report.ps1

Purpose: Read-only reporting for the "Scoping Application-Permission Mailbox Access" solution. Produces the security-review artifact: for every app / managed identity holding a mail-capable application Graph permission, one row stating the app, the permission(s), whether it is SCOPED (via an Application Access Policy / Exchange RBAC assignment) or TENANT-WIDE, and — when scoped — the mailboxes/scope it is allowed to reach. Emits a summary count of unscoped apps (the number you are

driving to zero). Makes no changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\MBXRBAC-Report.ps1 powershell -ExecutionPolicy Bypass -File .\MBXRBAC-Report.ps1 -CsvPath .\MBXRBAC-scoping-report.csv Requires Microsoft.Graph and ExchangeOnlineManagement. Run Connect-MgGraph and Connect-ExchangeOnline first, or the script prompts. REQUIRED SCOPES / ROLES Microsoft Graph (delegated): Application.Read.All, Directory.Read.All. Exchange Online: a View-Only role able to run Get-ApplicationAccessPolicy / Get-DistributionGroupMember. VERIFY exact scopes, roles, and app-role IDs against current Microsoft documentation. EXPECTED RESULTS A tenant-wide table + CSV of every mail-capable application permission and its scoping status, plus a "TENANT-WIDE apps remaining: N" summary. The CSV is the auditor's evidence. Read-only. .NOTES Azure Solutions — read-only. White-label sample values use contoso.com. All cmdlet/permission names: verify against current Microsoft documentation. > [CmdletBinding()] param([string]$CsvPath = ".\MBXRBAC-scoping-report.csv")

How to run: powershell -ExecutionPolicy Bypass -File .\MBXRBAC-Report.ps1 powershell -ExecutionPolicy Bypass -File .\MBXRBAC-Report.ps1 -CsvPath .\MBXRBAC-scoping-report.csv Requires Microsoft.Graph and ExchangeOnlineManagement. Run Connect-MgGraph and Connect-ExchangeOnline first, or the script prompts. REQUIRED SCOPES / ROLES Microsoft Graph (delegated): Application.Read.All, Directory.Read.All. Exchange Online: a View-Only role able to run Get-ApplicationAccessPolicy / Get-DistributionGroupMember. VERIFY exact scopes, roles, and app-role IDs against current Microsoft documentation. EXPECTED RESULTS A tenant-wide table + CSV of every mail-capable application permission and its scoping status, plus a "TENANT-WIDE apps remaining: N" summary. The CSV is the auditor's evidence. Read-only. .NOTES Azure Solutions — read-only. White-label sample values use contoso.com. All cmdlet/permission names: verify against current Microsoft documentation. > [CmdletBinding()] param([string]$CsvPath = ".\MBXRBAC-scoping-report.csv")

MBXRBAC-Set-AccessPolicy.ps1

Purpose: The single configuring script for the "Scoping Application-Permission Mailbox Access" solution. For the Application Access Policy model it ensures the mail-enabled security group exists, contains the allowed mailbox(es), and that an Application Access Policy binds the app to it. For the Graph RBAC-for-Exchange model it previews the scope + scoped role assignment. BY DEFAULT it runs in -WhatIf mode and only PRINTS what it would do; pass -Apply to execute. HOW TO RUN Preview (default — makes NO changes): powershell -ExecutionPolicy Bypass -File .\MBXRBAC-Set-AccessPolicy.ps1 `-AppId "11111111-1111-1111-1111-111111111111"` -ScopeGroup "App Mailbox Scope – Reporting – Prod" `-GroupSmtp "app-mailbox-scope-reporting@contoso.com"` -AllowedMailbox "reports@contoso.com" Execute for real (only when the preview is correct AND change control is approved): ... -Apply Requires ExchangeOnlineManagement. Run Connect-ExchangeOnline first, or the script prompts. REQUIRED SCOPES / ROLES Exchange Online: a role that can run New-DistributionGroup, Add-DistributionGroupMember, and New-ApplicationAccessPolicy (and, for the RBAC model, New-ManagementScope / New-ManagementRoleAssignment) — commonly Global Administrator or an Exchange role carrying Role Management / Organization Configuration rights. VERIFY the exact minimum role and all cmdlet names/parameters against current Microsoft documentation. EXPECTED RESULTS In -WhatIf (default): a printed plan of every object that WOULD be created, no changes made. With -Apply: the mail-enabled security group and Application Access Policy exist and scope the app; the script then reminds you to allow propagation and run MBXRBAC-Validate.ps1 + MBXRBAC-Test-Boundary.ps1. .NOTES Azure Solutions — STATE-CHANGER, defaults to -WhatIf. White-label sample values use contoso.com. Every cmdlet/parameter/role/model name below: VERIFY against current Microsoft documentation — exact syntax and which model is current/recommended are

version- and cloud-dependent. > [CmdletBinding()] param([Parameter(Mandatory)] [string]$AppId, [string]$ScopeGroup = "App Mailbox Scope – Reporting – Prod", [string]$GroupSmtp = "app-mailbox-scope-reporting@contoso.com", [string[]]$AllowedMailbox = @("reports@contoso.com"), [string]$PolicyDescription = "App Mailbox Scope – Reporting – Prod – Policy", [ValidateSet('RestrictAccess','DenyAccess')] [string]$AccessRight = 'RestrictAccess', [switch]$Apply # default OFF => -WhatIf preview only)

How to run: Preview (default — makes NO changes): powershell -ExecutionPolicy Bypass -File .\MBXRBAC-Set-AccessPolicy.ps1 `-AppId "11111111-1111-1111-1111-111111111111"` -ScopeGroup "App Mailbox Scope – Reporting – Prod" `-GroupSmtp "app-mailbox-scope-reporting@contoso.com"` -AllowedMailbox "reports@contoso.com" Execute for real (only when the preview is correct AND change control is approved): ... -Apply Requires ExchangeOnlineManagement. Run Connect-ExchangeOnline first, or the script prompts. REQUIRED SCOPES / ROLES Exchange Online: a role that can run New-DistributionGroup, Add-DistributionGroupMember, and New-ApplicationAccessPolicy (and, for the RBAC model, New-ManagementScope / New-ManagementRoleAssignment) — commonly Global Administrator or an Exchange role carrying Role Management / Organization Configuration rights. VERIFY the exact minimum role and all cmdlet names/parameters against current Microsoft documentation. EXPECTED RESULTS In -WhatIf (default): a printed plan of every object that WOULD be created, no changes made. With -Apply: the mail-enabled security group and Application Access Policy exist and scope the app; the script then reminds you to allow propagation and run MBXRBAC-Validate.ps1 + MBXRBAC-Test-Boundary.ps1. .NOTES Azure Solutions — STATE-CHANGER, defaults to -WhatIf. White-label sample values use contoso.com. Every cmdlet/parameter/role/model name below: VERIFY against current Microsoft documentation — exact syntax and which model is current/recommended are version- and cloud-dependent. > [CmdletBinding()] param([Parameter(Mandatory)] [string]$AppId, [string]$ScopeGroup = "App Mailbox Scope – Reporting – Prod", [string]$GroupSmtp = "app-mailbox-scope-reporting@contoso.com", [string[]]$AllowedMailbox = @("reports@contoso.com"), [string]$PolicyDescription = "App Mailbox Scope – Reporting – Prod – Policy", [ValidateSet('RestrictAccess','DenyAccess')] [string]$AccessRight = 'RestrictAccess', [switch]$Apply # default OFF => -WhatIf preview only)

MBXRBAC-Test-Boundary.ps1

Purpose: Read-only proof for the "Scoping Application-Permission Mailbox Access" solution. Using the application's OWN credential (client credentials for the scoped app registration / managed identity), it attempts a Microsoft Graph mail read against the in-scope mailbox and against the out-of-scope mailbox, and reports PASS/FAIL: in-scope read -> must SUCCEED (HTTP 200) out-of-scope -> must be DENIED (HTTP 403 / ErrorAccessDenied) This is the evidence that the scope actually constrains the app. Reads only; sends/writes no mail and changes no configuration. HOW TO RUN App-registration (client secret) example: powershell -ExecutionPolicy Bypass -File .\MBXRBAC-Test-Boundary.ps1 `-TenantId "<tenant-guid>"` -AppId "11111111-1111-1111-1111-111111111111" `-ClientSecret (Read-Host -AsSecureString "App client secret")` -InScopeMailbox "reports@contoso.com" -OutOfScopeMailbox "ceo@contoso.com" Managed identity (Logic App / Function / VM) — omit -ClientSecret and use -UseManagedIdentity. Requires Microsoft.Graph.Authentication. Prefer a certificate or managed identity over a client secret in production. REQUIRED SCOPES / ROLES The APPLICATION being tested must hold the mail application permission being proven (e.g. Mail.Read) — that is the point. The operator running the script needs nothing beyond the app's own credential. VERIFY exact Graph permission names and endpoints against current Microsoft documentation (and use the correct

national-cloud endpoint for GCC/GCC High/DoD). EXPECTED RESULTS PASS when the in-scope read returns 200 AND the out-of-scope read returns 403/denied. FAIL if the out-of-scope read succeeds (scope not in effect — wrong model, mis-scope, or propagation not complete) or the in-scope read is denied (scope too tight). Read-only. .NOTES Azure Solutions — read-only (mailbox reads only). White-label sample values use contoso.com. All permission names/endpoints: verify against current Microsoft documentation. > [CmdletBinding()] param([Parameter(Mandatory)] [string]$TenantId, [Parameter(Mandatory)] [string]$AppId, [System.Security.SecureString]$ClientSecret, [switch]$UseManagedIdentity, [string]$InScopeMailbox = "reports@contoso.com", [string]$OutOfScopeMailbox = "ceo@contoso.com")

How to run: App-registration (client secret) example: powershell -ExecutionPolicy Bypass -File .\MBXRBAC-Test-Boundary.ps1 `-TenantId "<tenant-guid>"` -AppId "11111111-1111-1111-1111-111111111111" `-ClientSecret (Read-Host -AsSecureString "App client secret")` -InScopeMailbox "reports@contoso.com" -OutOfScopeMailbox "ceo@contoso.com" Managed identity (Logic App / Function / VM) — omit -ClientSecret and use -UseManagedIdentity. Requires Microsoft.Graph.Authentication. Prefer a certificate or managed identity over a client secret in production. REQUIRED SCOPES / ROLES The APPLICATION being tested must hold the mail application permission being proven (e.g. Mail.Read) — that is the point. The operator running the script needs nothing beyond the app's own credential. VERIFY exact Graph permission names and endpoints against current Microsoft documentation (and use the correct national-cloud endpoint for GCC/GCC High/DoD). EXPECTED RESULTS PASS when the in-scope read returns 200 AND the out-of-scope read returns 403/denied. FAIL if the out-of-scope read succeeds (scope not in effect — wrong model, mis-scope, or propagation not complete) or the in-scope read is denied (scope too tight). Read-only. .NOTES Azure Solutions — read-only (mailbox reads only). White-label sample values use contoso.com. All permission names/endpoints: verify against current Microsoft documentation. > [CmdletBinding()] param([Parameter(Mandatory)] [string]$TenantId, [Parameter(Mandatory)] [string]$AppId, [System.Security.SecureString]$ClientSecret, [switch]$UseManagedIdentity, [string]$InScopeMailbox = "reports@contoso.com", [string]$OutOfScopeMailbox = "ceo@contoso.com")

MBXRBAC-Validate.ps1

Purpose: Read-only validation for the "Scoping Application-Permission Mailbox Access" solution. For a named application (AppId) and its intended mailbox scope, confirms that an Exchange Application Access Policy (or an Exchange RBAC assignment) actually covers the app, that the mail-enabled security group is the correct type and contains only the allowed mailboxes, and that Microsoft's built-in policy test grants the in-scope mailbox and denies an out-of-scope one. Emits a single PASS/FAIL. Makes no changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\MBXRBAC-Validate.ps1 `-AppId "11111111-1111-1111-1111-111111111111"` -ScopeGroup "App Mailbox Scope – Reporting – Prod" `-InScopeMailbox "reports@contoso.com"` -OutOfScopeMailbox "ceo@contoso.com" Requires ExchangeOnlineManagement. Run Connect-ExchangeOnline first, or the script prompts. REQUIRED SCOPES / ROLES Exchange Online: a role that can run Get-ApplicationAccessPolicy, Test-ApplicationAccessPolicy, and Get-DistributionGroup / Get-DistributionGroupMember (View-Only Recipients / View-Only Configuration are typically sufficient). VERIFY exact role and cmdlet names against current Microsoft documentation. EXPECTED RESULTS PASS when: a policy/assignment covers the AppId; the scope group is a mail-enabled security group containing the in-scope mailbox and NOT the out-of-scope mailbox; the built-in test grants in-scope and denies out-of-scope. Otherwise FAIL with the reason. Read-only. .NOTES Azure Solutions — read-only. White-label sample values use contoso.com. All cmdlet names: verify against current Microsoft documentation. > [CmdletBinding()] param(

[Parameter(Mandatory)] [string]$AppId, [string]$ScopeGroup = "App Mailbox Scope – Reporting – Prod", [string]$InScopeMailbox = "reports@contoso.com", [string]$OutOfScopeMailbox= "ceo@contoso.com")

How to run: powershell -ExecutionPolicy Bypass -File .\MBXRBAC-Validate.ps1 `-AppId "11111111-1111-1111-1111-111111111111"` -ScopeGroup "App Mailbox Scope – Reporting – Prod" `-InScopeMailbox "reports@contoso.com"` -OutOfScopeMailbox "ceo@contoso.com" Requires ExchangeOnlineManagement. Run Connect-ExchangeOnline first, or the script prompts. REQUIRED SCOPES / ROLES Exchange Online: a role that can run Get-ApplicationAccessPolicy, Test-ApplicationAccessPolicy, and Get-DistributionGroup / Get-DistributionGroupMember (View-Only Recipients / View-Only Configuration are typically sufficient). VERIFY exact role and cmdlet names against current Microsoft documentation. EXPECTED RESULTS PASS when: a policy/assignment covers the AppId; the scope group is a mail-enabled security group containing the in-scope mailbox and NOT the out-of-scope mailbox; the built-in test grants in-scope and denies out-of-scope. Otherwise FAIL with the reason. Read-only. .NOTES Azure Solutions — read-only. White-label sample values use contoso.com. All cmdlet names: verify against current Microsoft documentation. > [CmdletBinding()] param([Parameter(Mandatory)] [string]$AppId, [string]$ScopeGroup = "App Mailbox Scope – Reporting – Prod", [string]$InScopeMailbox = "reports@contoso.com", [string]$OutOfScopeMailbox= "ceo@contoso.com")